

myCobot API

1 Prerequisite for controlling the robotic arm

1.1 MyCobot (string port, int baud=115200)

Function: Instantiate MyCobot

Return value: None

Parameter Description: Parameter 1: Serial Port Number ("COM *" on Windows (such as COM30)

Parameter 2: Baud Rate (default is 115200)

Note: **If you want to call the following API, you need to first instantiate**

1.2 Open()

Function: Open serial port

Return value: None

Parameter Description: None

Attention: **To communicate with the robotic arm, you need to first open the serial port**

1.3 Close()

Function: Close serial port

Return value: None

Parameter Description: None

Note: **It is best to close the serial port at the end of the program**

Overall Status of 2 Robots

2.1 PowerOn();

Function: Power on the robotic arm

Return value: None

Parameter Description: None

Note: **After powering on the robotic arm, it will not be possible to manually move the arm**

2.2 PowerOff()

Function: The robotic arm loses power

Return value: None

Parameter Description: None

Attention: **After powering on the robotic arm, if you want to manually move it, you can use this API to make the robotic arm lose power**

3 Input Program Control Mode MDI Mode and Robot Control (Manual Data Input)

3.1 SendOneAngle(int jointNo, int angle, int speed)

Function: Send single joint angle

Return value: None

Parameter Description: Parameter 1: Joint Number (**1-6**), Parameter 2: Angle (Range: **-170 ° -170 °**),

Parameter 3: Speed (**0-100**)

Use case: [Click to view](#).

3.2 **GetAngles()**

Function: Obtain all joint angles

Return value: Returns an int type array, int [], length: 6

Parameter Description: None

Use case: [Click to view](#).

3.3 **SendAngles (int [] angles, int speed)**

Function description: Send all joint angles

Return value: None

Parameter Description: Parameter 1: All joint angles (range: **-170 ° -170 °**), Parameter 2: Speed (**0-100**)

Use case: [Click to view](#).

3.4 **GetCoords()**

Function: Obtain all coordinates

Return value: Returns an int type array, int [], length: 6

Parameter Description: None

Use case: [Click to view](#).

3.5 **SendCoords(int[] coords, int speed, int mode)**

Function: Send multi parameter coordinates

Return value: None

Parameter Description: Parameter 1: All coordinates (X, Y, and Z values range from -300 to 300.00 units in mm RX, RY, RZ, value range **-180**), Parameter 2: Speed (**0 to 100**), Parameter 3: Mode (**0- angular, 1- linear**)

Use case: [Click to view](#).

3.6 **SendOneCoord(int coord, int value, int speed)**

Function: Send single parameter coordinates

Return value: None

Parameter Description: Parameter 1: Coordinate Number (1-6 (**x, y, z, rx, ry, rz**)), Parameter 2: Coordinate (X, Y, Z value range -300-300.00 units mm RX, RY, RZ, value range **-180-180**), Parameter 3: Speed (**0-100**)

Use case: [Click to view](#)

4 Atom End IO control Atom IO Control

4.1 **SetDigitalOut(byte pin_number, byte pin_signal)**

Function: Set the high and low levels of output IO

Return value: None

Parameter Description: Parameter 1: Pin Number (Atom Output Pin Number), Parameter 2: State (0- Low Level, 1- High Level)

Use case: [Click to view](#)

4.2 **GetDigitalIn(byte pin_number)**

Function: Get input IO status

Return value: Pin status (0- low level, 1- high level)

Parameter Description: Pin Number (Atom Input Pin Number)

Use case: [Click to view](#)

4.3 **SetGripperMode(byte mode)**

Function: Set Gripper Mode

Return value: None

Parameter Description: Parameter 1: mode:1-transparent mode, 0-IO mode

4.4 **setGripperValue(byte angle, byte speed)**

Function: Control adaptive gripper

Return value: None

Parameter Description: Parameter 1: Claw opening and closing angle (0-100, 0- closed, 100- maximum opening angle), Parameter 2: Claw opening and closing speed (0-100)

Use case: [Click to view](#)

4.5 **SetElectricGriper (byte open)**

Function: Control electric gripper

Return value: None

Parameter description: Claw switch status (0- off, 1- on)

4.6 **getGripperValue ()**

Function: Obtain adaptive gripper angle

Return value: int type, returns the gripper angle (0- closed, 100- maximum open angle)

Parameter Description: None

Use case: [Click to view](#)

5 Base M5Stack basicIO control M5Stack-basic IO Control

5.1 **SetBasicOut(byte pin_number, byte pin_signal)**

Function: Set the high and low levels of output IO

Return value: None

Parameter Description: Parameter 1: Pin Number (Basic Output Pin Number), Parameter 2: State (0- Low Level, 1- High Level)

Use case: [Click to view](#)

5.2 **GetBasicIn(byte pin_number)**

Function: Get input IO status

Return value: Pin status (0- low level, 1- high level)

Parameter Description: Pin Number (Basic Input Pin Number)

Use case: [Click to view](#)